CONCEPTION PHASE:

- Task: Create a Habit Tracking App:

Introduction:

The objective is to create an effectively designed, scalable Python habit tracking tool that manages weekly and daily habits, keeps data in storage, provides analytics, and utilizes a basic command-line interface [CLI] for an efficient, clear and easily navigable interface. For maximum maintainability and scalability, the design adheres to object-oriented principles for data modelling underneath and functional programming for analytics. The application is simple to test, scale, and maintain due to the fact that it is designed in a modular form since it takes advantage of SQLite for scaling and support of multiple users and CLI which is simple to test without needing the use of GUI elements. The utilization of object-oriented and functional programming paradigms maximizes the advantages of Python and also fulfills project goals.

1. System Architecture:

- Habit Class:

The habit class is a core component where users create habits. It contains all the necessary information to create a single habit. It uses attributes such as habit_name for naming the habits, period for the time-period a habit persists, created_at for the time when the habit was created, and log_history for the analysis such as how long a habit has been maintained for. The habit class uses methods such as mark_completed() for when the habit is completed, get_streak() to check if the habit is due.

- HabitManager Class:

The HabitManager class is a critical component of the habit tracker. Its purpose is to oversee the creation, tracking, updating, and deletion of habits for users. It controls the main functions like create_habit() that creates a new habit with a specified name and duration[like daily, weekly or even monthly], delete_habit() that deletes the currently selected habit, get_habits() that gets a list of all the habits that are currently being tracked, get_habits_by_period() that returns the habits based on their duration, and get_longest_streak() that returns the habits of the longest completion streak.

- StorageHandler Class:

This StorageHandler class operates between the user interface of the application and the actual data storage, ensuring that all habit-related data is reliably recorded, retrieved, and updated as per the user's input. The methods this class follows include: get_habits() retrieves all the existing habits from the database, create_habit() adds new habits to the database, delete_habit() removes existing habits from the database, get_habits_by_period() returns habits from the database after filtering it by the period, and get_longest_streak() returns the habits from the database with longest completion streak.

- Analytics Class:

For a habit tracker to transform habit data into clear insights and provide users with a useful feedback, the habit tracker must include an analytical class. The statistics and insights provided by this class regarding the habits of the users is based on the data that includes streaks, performance over time, and patterns in habit completion. In order to assist users to understand and improve their habits, this class usually works with the habit data, which is frequently stored in a database or a CSV file.

2. User Interaction and Flow:

- Startup: Loads previously stored habits from storage
- Main Menu: User may select to: create a habit, mark done, analyze habits and delete a habit.
- Creating Habits: User must input name and duration; habit is created and stored
- Completing Habits: User must select habit and mark as done for the day/week
- Analytics: User can see all habits, filter by type and see streaks
- Shutdown: All changes are saved prior to exit

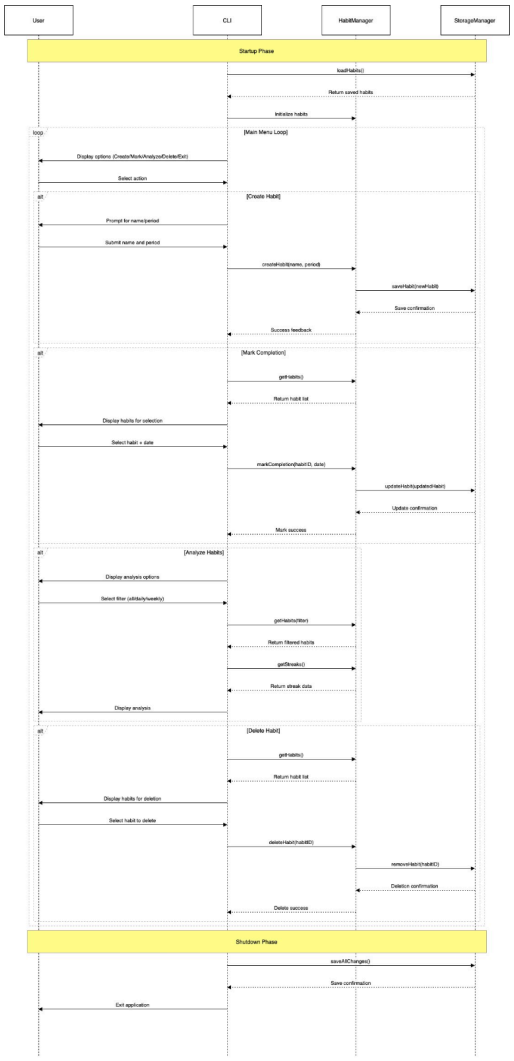
3. Design Decisions:

- OOP for Data Modelling: Object-Oriented Programming[OOP] is a fundamental paradigm in Python for data modeling and structuring an application . It can be employed to simulate real-

world objects as Python classes, having their attributes [data] and actions [methods] encapsulated.

- Data Storage: SQLite[Python sqlite3 module] is chosen because it is best suited to embed database functionality within Python software and accommodate complex logic, automation and integration with other Python applications.
- Functional Programming for Analytics: Analytics are computations based on data and functional programming in Python as it is a powerful approach for analytics since it facilities clean, maintainable and scalable data processing by leveraging functions, immutability and ensures easier testing.
- CLI for user interaction: CLI ensures accessibility for all users without GUI dependencies which makes it easy to automate and test as it requires minimal memory, processing power and are generally a stable tool which is crucial in the long run.

4. UML Diagram:

The UML diagram that illustrates a user's step-by-step interaction with the habit tracker app is included below. It demonstrates how a user can create a habit, mark it as complete, check analytics for the habit they have chosen, and, if they decide to, can delete the habit. All of the changes made when the user logged in are saved by the app when the user exits the app. Below mentioned is the function-interaction of the Habit tracker app:

- HabitManager manages multiple Habits.
- StorageLayer is used by HabitManager to save/load data
- Analytics operates on the data provided by HabitManager
- CLI acts as an interface and interacts with HabitManager and Analytics.
- Below Mentioned is a Sequence Diagram for the exemplary workflow of this habit tracker.